



University of Pisa

Department of Information Engineering

Design and Secure Deployment of Hierarchical Infrastructures: The ODA Case Study

Candidate: Giovanni Neglia

Master's Degree in Cybersecurity

Academic Year 2025/2026

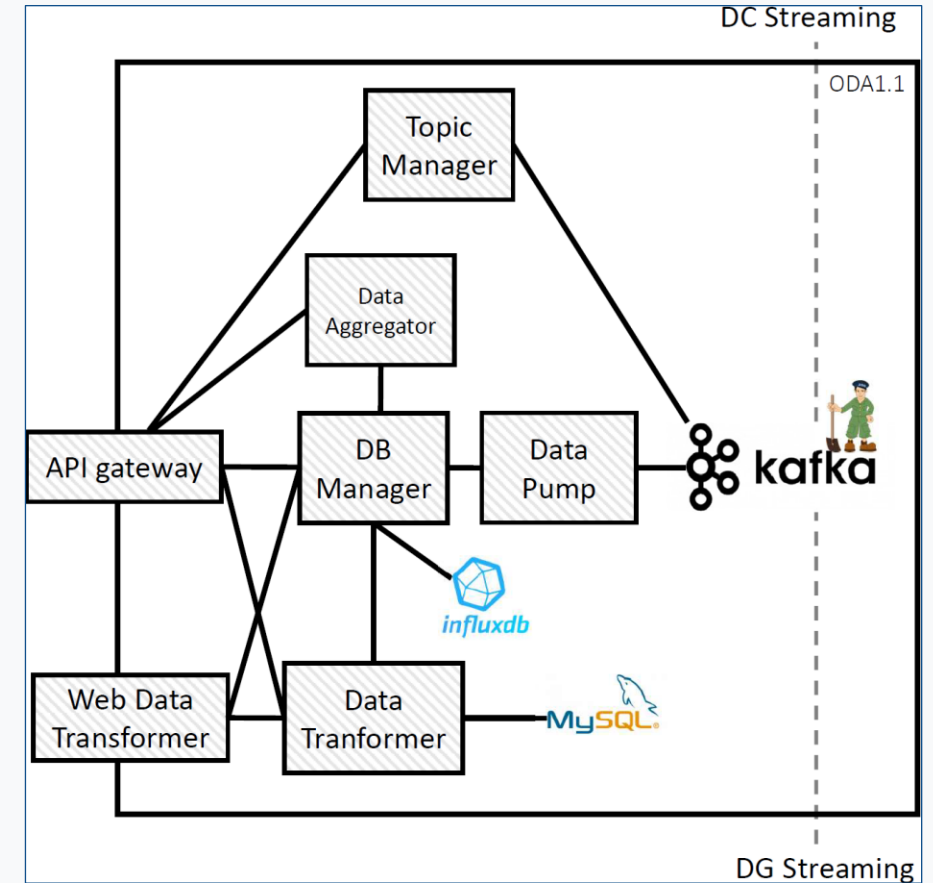
Supervisors:

Prof. Antonio Brogi
Dott. Alessandro Bocci
Prof. Gian-Luigi Ferrari

I. Research Context and Objectives

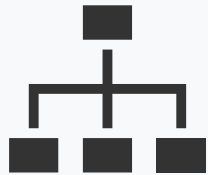
Context & Motivation

- **Industry 4.0 Paradigm Shift:** Evolution from centralized cloud infrastructures towards distributed edge computing.
- **Hierarchical Data Streaming:** Real-time applications require Edge-to-Cloud architectures to scale dependably.
- **The Case Study:** Observable Data Access (ODA) handles localized data ingestion via microservices.
- **Limitations:** ODA operates as an isolated standalone instance and lacks security measures.



Objectives & Contribution Overview

Main Goal: Design and secure a hierarchical Edge-to-Cloud deployment for the ODA ecosystem.



Hierarchical Design:

Defining and evaluating distinct data replication topologies.



Macro-Architectural Threat Modeling:

Formalizing trust boundaries and mapping threats.



Secure by Design PoC: Utilizing Ansible as a declarative Security-as-Code engine.

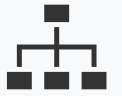


Results Validation:

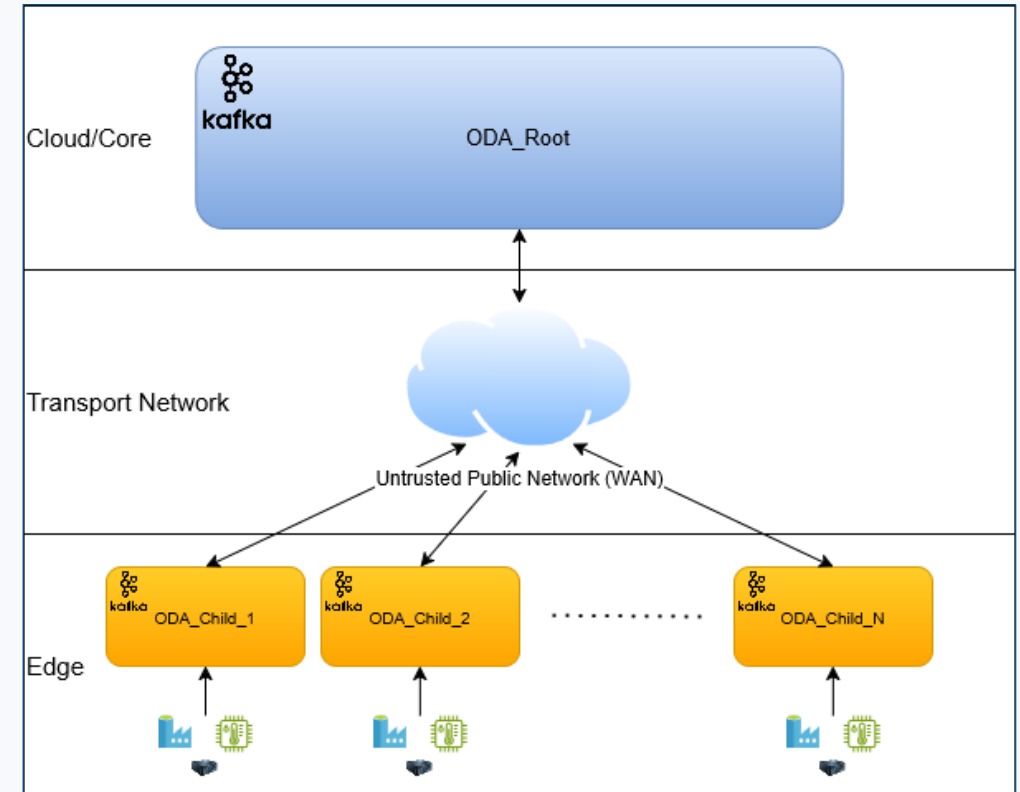
Assessing the infrastructure's resilience.

II. Hierarchical ODA Design

Bridging the Gap: Hierarchical ODA

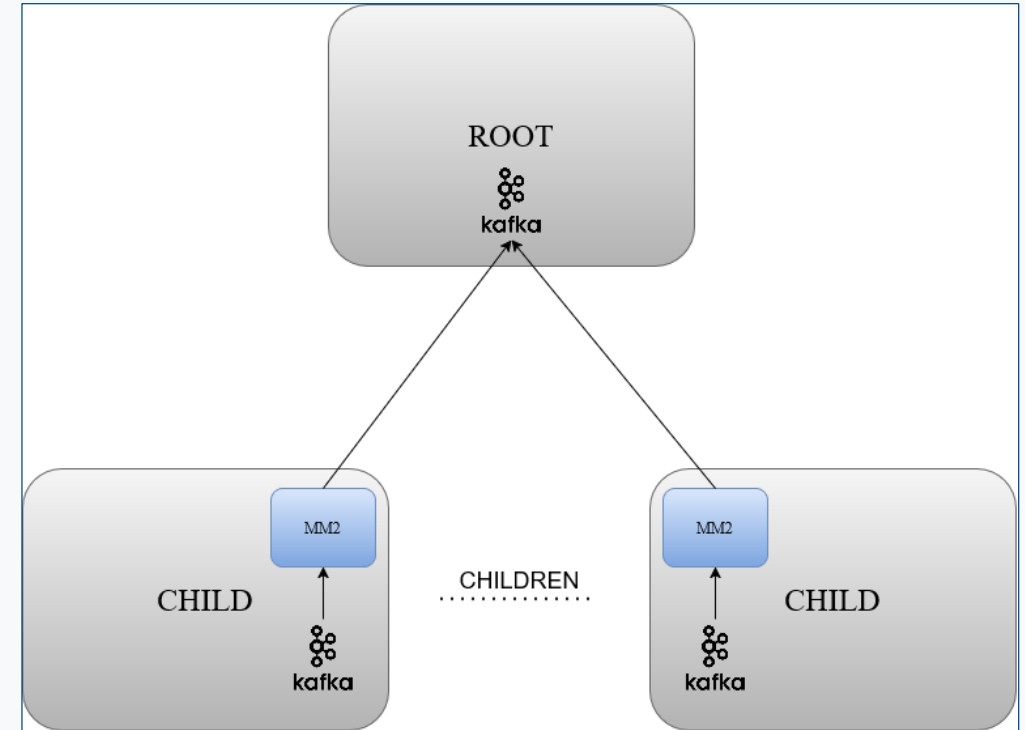


- **Multi-Tier Ecosystem:** Evolving ODA requires connecting geographically dispersed Child nodes to a central Root over an untrusted transport network.
- **Replication Engine:** MirrorMaker 2 (MM2) is selected to bridge the isolated brokers and ensure cross-node data synchronization.
- **Architectural Fork:** The deployment strategy changes depending on where the MM2 engine is located.



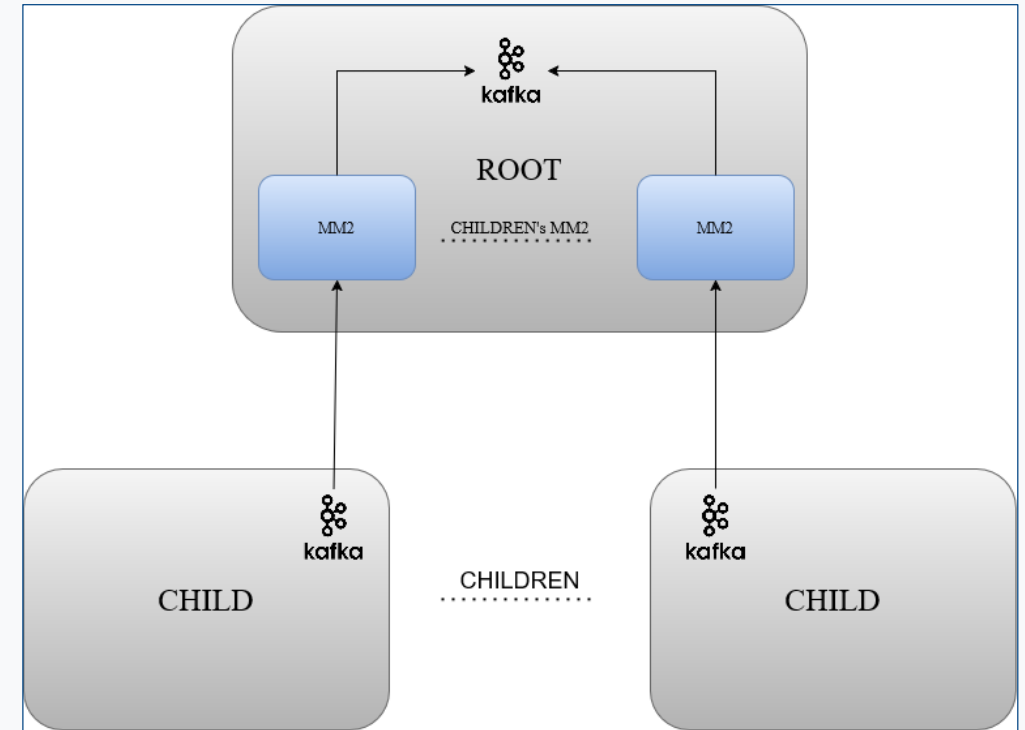
The Child-Push Topology

- **Network Advantage:** Edge initiates egress traffic, natively traversing NAT and stateful firewalls.
- **Core Exposure:** The central Root broker must expose a public endpoint to the untrusted transport network.
- **Decentralized Governance:** MM2 runs locally; therefore, topic replication logic is managed at the Edge.



The Root-Pull Topology

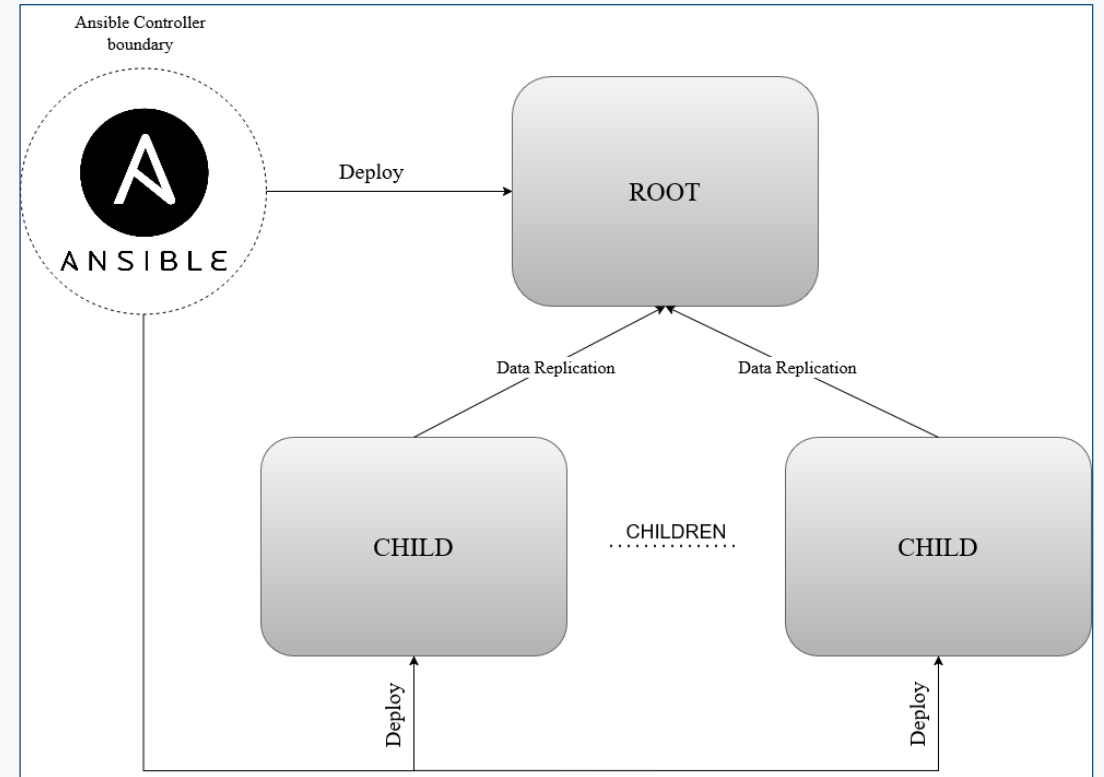
- **Core Protection:** The Root initiates only outbound connections, keeping the central Kafka broker hidden.
- **Centralized Governance:** Replication logic is managed entirely at the Root; compromised nodes can be instantly quarantined.
- **Edge Exposure:** Every peripheral Child node must expose its broker, requiring a hardened listening posture.
- **Logistical & Orchestration Complexity:** Requires VPN overlays to reach the Edge, and dynamic 1-to-N MM2 container scaling on the Root.



III. Threat Modeling

System Modeling & Trust Boundaries

- **System Model:** The DFD maps interactions to enforce trust boundaries.
- **"Black Box" Constraint:** baseline ODA stack lack native security and is inherited as an encapsulated secure entity.
- **Trusted Components (Out of Scope):** Ansible Controller, underlying hardware/hypervisor, and infrastructure-level network filtering are out of scope.
- **Zero Trust Environments (In Scope):** The deployed Root and Child nodes, alongside the transport network (Deploy and Data Replication flow), are untrusted.



Profiling the Adversaries



- **Hybrid Methodology:** Integrating STRIDE categories with MITRE ATT&CK techniques to model three distinct adversarial profiles.



[T-NA] Network

Adversary: Operates on the untrusted transport network to execute Sniffing and MitM attacks.



[T-MO] Malicious

Observer: Has unprivileged shell access, performing local reconnaissance to harvest secrets.



[T-CA] Compromised

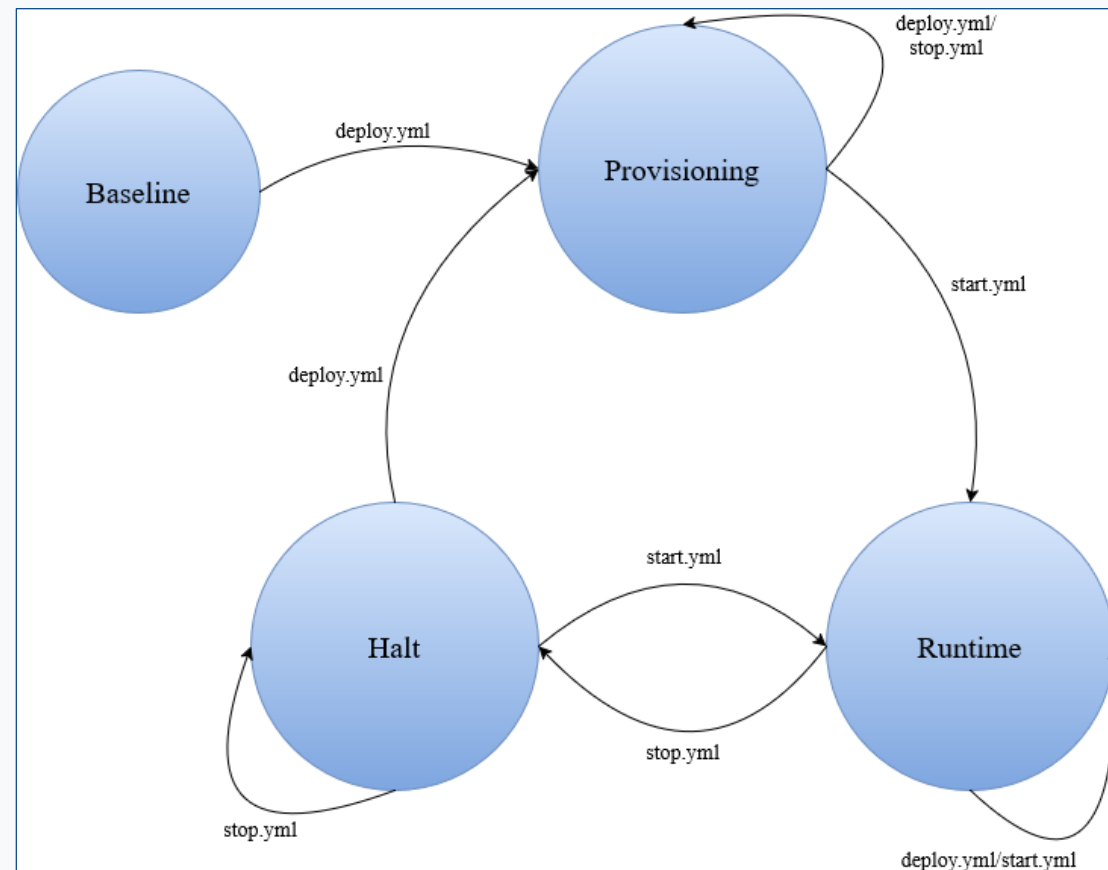
Administrator: Achieves full root privileges to sabotage data pipelines.

IV. PoC Implementation and Validation

Playbook Orchestration



- **The IaC Paradigm:** Transitioning from manual setups to a declarative IaC methodology for scalable and secure deployments.
- **Lifecycle Separation:** Deployment is divided into isolated operational phases to prevent runtime collisions.



Mitigations: Transport Security



- **Deploy Flow Protection:** Enforcing Strict SSH Host Key Verification to secure the Ansible management channel.
- **Data Replication Flow Protection:** Enforcing Mutual TLS (mTLS) to cryptographically authenticate and encrypt all inter-node traffic.
- **Automated PKI & Distribution:** A dedicated script dynamically generates certificates, which Ansible distributes adhering to the principle of least privilege.

```
## mirrormaker 2 configuration

# cluster names
clusters={{ inventory_hostname }},root

# source aka our child, pushing data
{{ inventory_hostname }}.bootstrap.servers=kafka_push:{{ kafka_internal_port }}

# target aka our root
root.bootstrap.servers={{ root_kafka_address }}:{{ root_kafka_port }}

## root SSL config
root.security.protocol=SSL
root.ssl.truststore.location={{ kafka_secrets_dir }}/kafka.client.truststore.jks
root.ssl.truststore.password={{ kafka_ssl_password }}
root.ssl.keystore.location={{ kafka_secrets_dir }}/kafka.client.keystore.jks
root.ssl.keystore.password={{ kafka_ssl_password }}
root.ssl.key.password={{ kafka_ssl_password }}
```

Mitigations: Secret and System Hardening



- **Ansible Vault Encryption:** Sensitive variables are dynamically decrypted only during the deployment execution to populate templates.
- **POSIX 0700 Hardening:** Enforcing a strict "Default Deny" boundary on the host to lock out unprivileged local observers.

```
# keystore and truststore password (encrypted)
kafka_ssl_password: !vault |
    $ANSIBLE_VAULT;1.1;AES256
    3732643838323231383861363466623737363
    3431353337623363356366616337326339316
    6434613965393135306634393962326364333
    3461663766616438310a386564343239626430
    6365
```

```
- name: create oda base deployment directory
  file:
    path: "{{ oda_deploy_path }}"
    state: directory
    mode: '0700'
```

Mitigations: Replication Governance



- **Identity-Based Governance:** Dynamically translating verified mTLS identities into granular Kafka Access Control Lists (ACLs).
- **Blast Radius Containment:** Enforcing strict topic prefixing to prevent tenant impersonation and unauthorized cross-tenant data access.
- **Endpoint DoS Mitigation:** Applying bandwidth quotas (1 MB/s) to throttle traffic and mitigate resource exhaustion attacks.

```
- name: acl business topics
  community.docker.docker_container_exec:
    container: kafka_push
    command: >
      /kafka/bin/kafka-acls.sh --bootstrap-server localhost:{{ kafka_internal_port }}
      --add
      --allow-principal User:CN={{ item }}
      --producer
      --topic "{{ item }}.testingtopic"
      --resource-pattern-type PREFIXED
  loop: "{{ groups['child'] }}"
```


Mitigations: Drift Control



- **Continuous State Enforcement:**
Scheduled playbook detects MM2 configuration drift, reverting file alterations to the trusted baseline.
- **Event-Driven Handlers:** Ansible dynamically triggers a selective restart of the affected MM2 container, avoiding infrastructure-wide restarts.

```
- name: check if mm2 container is currently running
  command: docker ps -q -f name=mirrormaker_push
  register: mm2_status
  changed_when: false
  listen: restart mm2 container

- name: restart mm2 container
  command: docker restart mirrormaker_push
  when: mm2_status.stdout != ""
  listen: restart mm2 container
```

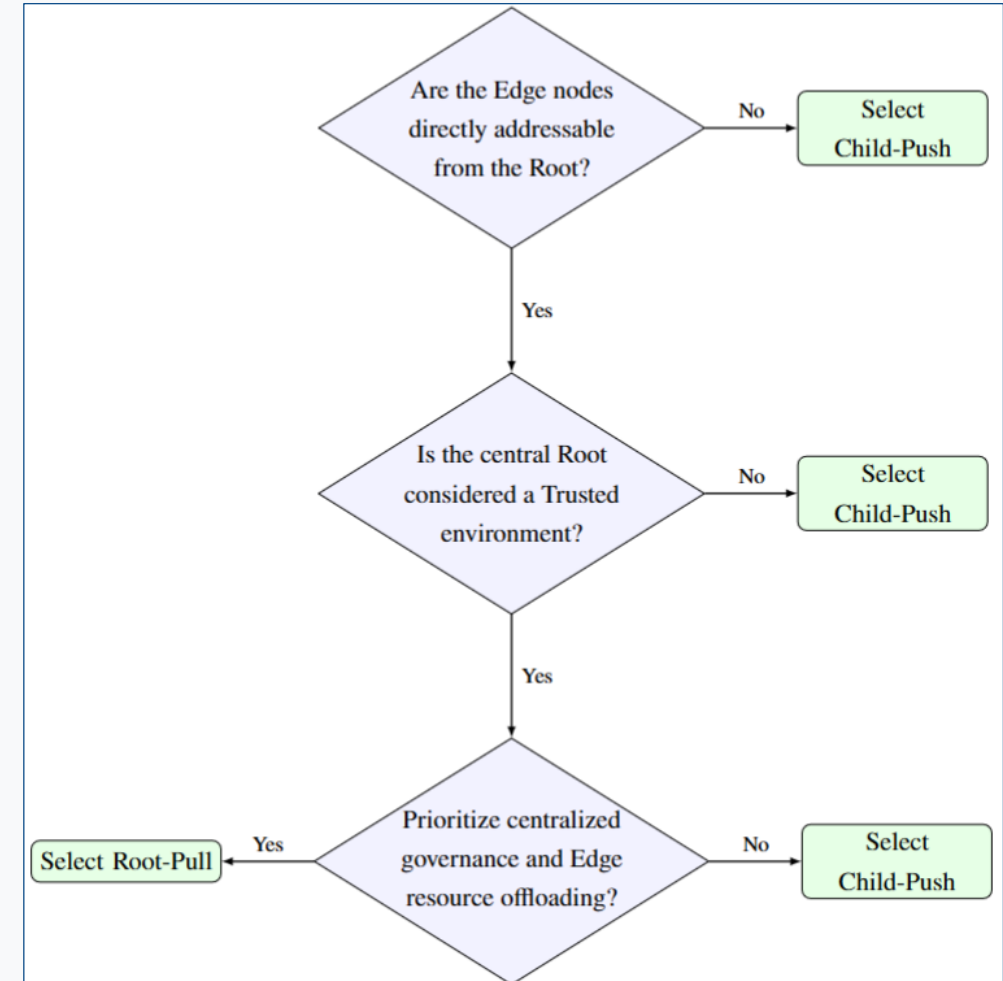
Security Validation Outcomes



- **Live VPN Testbed:** Security countermeasures were actively assessed on a realistic, distributed VM infrastructure.
- **Network Adversary Mitigated:** mTLS handshakes successfully dropped all unauthenticated connection attempts.
- **Malicious Observer Locked Out:** Strict POSIX boundaries blocked unprivileged local secrets harvesting.
- **Compromised Admin Contained:** Dynamic ACLs and bandwidth quotas thwarted data poisoning and simulated resource exhaustion attacks.

Architectural Decision Tree

- **No Silver Bullet:** Architecture selection is dictated by network constraints, trust assumptions, and specific industrial logistics.



V. Conclusions

Conclusions & Future Work

- **Objective Achieved:** Successfully bridged the ODA Edge-to-Cloud continuum enforcing a Secure by Design paradigm via Security-as-Code.
- **Known limitations and possible enhancements:**
 - **Internal Microservices Security:** Native authentication and input validation within ODA components.
 - **Logging & Non-Repudiation:** Implementation of tamper-evident auditing for full traceability.
 - **Advanced Remediation:** Evolving event-driven logic for complex state-evaluation workflows.
 - **Network Perimeter:** Integration with enterprise-grade hardware firewalls.
 - **Granular Secret Management:** Granular passphrase generation for individual assets.

**Thank You for Your
Attention**

A. Backup Slides

Replication Performance

- **The Cost of Security:** Continuous mTLS cryptographic encapsulation constitutes the primary End-to-End throughput bottleneck.
- **Edge Viability:** The minimal memory and CPU footprint confirms that both architectures can be reliably deployed on resource-constrained nodes.

Metric	Child-Push Topology	Root-Pull Topology
Workload	500,000 rec. (1 KB)	500,000 rec. (1 KB)
Local Edge Ingestion	23.92 MB/s	25.85 MB/s
E2E Replication Throughput	~1.93 MB/s	~1.84 MB/s
MM2 Avg CPU	~4–8%	~4–7%
MM2 Max RAM	~379 MiB	~332 MiB